

Lab 7: Vulnerability Analysis — Extended

Systematic Scanning, Credentialed Assessment, Business-Context Triage, and Remediation Verification

Course: Hands-On Ethical Hacking and Network Defense — Vulnerability Analysis module companion lab **Estimated time:** 4–5 hours — can span two sessions **Format:** Individual; uses your isolated host-only lab (Kali attacking Metasploitable 2 and OWASP BWA) **Prerequisite:** Labs 1–4. A `baseline-pre-attack` snapshot of both targets available.

1. Learning Objectives

By the end of this session you will be able to:

- Explain what a vulnerability scanner actually does differently from the manual Nmap work in earlier sessions
- Run a web-focused vulnerability scan with Nikto and a full-stack scan with OpenVAS/Greenbone
- Compare credentialed and uncredentialed scanning, and explain why they produce different results
- Read a CVSS score and use it to triage findings instead of treating every result as equally urgent
- Tell a true finding from a false positive by investigating it, not just by assuming the scanner is right
- Re-prioritize the same findings differently depending on business context — the same list of vulnerabilities doesn't mean the same thing at every organization
- Fix a finding and prove the fix worked with a verification rescan
- Write a vulnerability assessment report distinct in structure from the penetration-test reports you've written so far

2. ⚠ Scope Reminder

Same rule as the rest of the series: everything below targets your own Metasploitable 2 and OWASP BWA VMs on your host-only network. Vulnerability scanners are noisy by design — they intentionally throw a large number of probes at a target to see what answers back — so running one against anything outside your lab without separate written authorization is a real problem, not a theoretical one.

3. A Word on OpenVAS and Internet Access

Parts D and E use OpenVAS/Greenbone, which needs a one-time internet-connected sync to download its vulnerability feed before it can scan anything — this is the one point in this lab where your attack VM needs to briefly leave isolation. Do this step once, before you start the graded portion of the lab: connect Kali to the internet (NAT adapter), run the sync, then switch back to host-only before scanning. If your lab time is short, Parts D and E are marked optional — Parts B, C, and everything from Part F onward teach the core skills using tools and techniques that need no extra setup.

4. What You'll Need

- Kali, Metasploitable 2, and OWASP BWA running and mutually reachable
- Nikto (pre-installed on Kali)
- OpenVAS/Greenbone Community Edition (optional — see Section 3)
- SSH access to Metasploitable 2 (the `msfadmin` / `msfadmin` default account works for this lab's credentialed-scan and remediation steps)
- Your Lab 2–4 notes (scan results, the vsftpd finding, your exploitation log)

5. Procedure

Part A — What a Scanner Does That Nmap Doesn't

1. Reread your Lab 2 Nmap output for Metasploitable. It told you *what's listening* and, with `-sV`, roughly *what version*.
2. A vulnerability scanner goes one step further: it checks each identified service and version against a database of **known** vulnerabilities and assigns each match a severity score. Nmap tells you a door is there; a scanner tells you that door's lock model has a known bypass, and ranks how bad that is.
3. Look up what **CVSS** (Common Vulnerability Scoring System) measures: record the three qualitative bands most scanners report findings in (commonly Low, Medium, High, Critical) and the rough numeric ranges each covers.

Part B — Web Vulnerability Scanning with Nikto

1. Run Nikto against OWASP BWA:

```
nikto -h http://<owasp-bwa-ip>
```

2. Run it against Metasploitable's web service as well:

```
nikto -h http://192.168.56.102
```

3. From the output, record **five** findings. For each: what Nikto flagged, and your own quick guess at its severity before you see any formal scoring.
4. Note any finding that references a specific **OSVDB** or **CVE** identifier — that's your hook into looking up the full vulnerability detail independently.

Part C — Interpreting and Triaging Nikto's Output

1. Pick your three most severe-looking findings from Part B.
2. For each, search its identifier (or description) to find its actual severity classification.
3. Build a triage table: finding, Nikto's raw description, severity band, and one sentence on exploitability — is this something you could act on with tools from earlier labs, or does it need something you don't have yet?
4. **Reflection (2-3 sentences):** Nikto flags some findings that are informational rather than exploitable (a banner disclosure, a missing security header). Why does a scanner report these at all if they're not directly exploitable?

Part D — Full-Stack Scanning with OpenVAS/Greenbone (Optional — See Section 3)

1. After completing the one-time feed sync (Section 3), launch the Greenbone web interface and log in.
2. Create a new target for Metasploitable 2's IP address, and a new task using a **Full and fast** scan config.
3. Run the scan. This can take 15-30 minutes even against a small target — use the wait time to start Part F with your Nikto data.
4. Once finished, open the results and record your top five findings by severity, including each one's CVSS score.

Part E — Credentialed vs. Uncredentialed Scanning (Optional — See Section 3)

1. Your Part D scan ran **uncredentialed** — OpenVAS saw Metasploitable only the way an outside attacker would, from the network.
2. In Greenbone, add an SSH credential for Metasploitable using the `msfadmin/msfadmin` account, then create a new target and task using that credential.
3. Run the credentialed scan and compare its results against Part D's: record how many **additional** findings appeared — these are typically local-only issues (missing OS patches, weak file permissions, outdated installed packages) that an uncredentialed network scan can't see at all.
4. **Reflection (2-3 sentences):** A client might refuse to provide credentials for a scan, treating it as a bigger access grant than they're comfortable with. What's lost by only ever scanning uncredentialed, and how would you make the case for credentialed scanning to a nervous client?
5. Switch Kali back to host-only networking now that you're done with OpenVAS for this lab.

Part F — Cross-Validate Against What You Already Know

1. From either Part B/C (Nikto) or Part D/E (OpenVAS), pick the finding related to the **vsftpd** service you exploited back in Lab 3.
2. Compare the scanner's severity rating for that finding against what you know actually happened when you exploited it — an unauthenticated remote root shell.
3. Record whether the scanner's rating matched the real-world impact, undersold it, or oversold it.
4. Pick one **other** finding that you have **no** earlier lab evidence for. Decide: would you trust this finding as-is, or flag it for manual verification before it goes in a report? Justify your answer in 1-2 sentences.

Part G — Scenario: Investigating a Suspected False Positive

Your scan results include a finding that a specific service is running an outdated, vulnerable version. When you look closer, something feels off — the banner the scanner relied on might not tell the whole story.

1. Pick one finding from Parts B-E that's based purely on a version string or banner (not something you've already confirmed by exploiting it).
2. Manually investigate it using tools from earlier labs: grab the banner directly with `nc -nv <target> <port>`, and check whether the service behaves the way the reported vulnerability would predict.
3. Reach a verdict: **confirmed**, **false positive**, or **inconclusive without deeper access**. Document your reasoning — what specifically did you check, and what would change your verdict?
4. **Reflection (1-2 sentences):** Some vendors backport security fixes without changing the version string a scanner reads. Why does that make version-based detection alone an unreliable way to confirm a vulnerability?

Part H — Scenario: Remediation and Rescan

You've reported the vsftpd finding. The "administrator" (you, wearing a different hat) has decided to remediate by disabling the vulnerable service rather than patching it — a common real-world shortcut when a patched version isn't readily available.

1. SSH into Metasploitable:

```
ssh msfadmin@192.168.56.102
```

2. Stop the vulnerable service:

```
sudo service vsftpd stop
```

3. From Kali, rescan the target with Nikto and/or Nmap's version detection and confirm the finding no longer appears.
4. Document this as a **remediation verification**: finding, action taken, rescan evidence, and the date/time you confirmed it. This is the exact record a real vulnerability management program keeps to prove a finding was actually closed, not just reported.
5. Restart the service and restore Metasploitable to its `baseline-pre-attack` snapshot before moving on, so the rest of the lab runs against the original target state.

Part I — Score and Prioritize

1. Take every finding you've recorded across Parts B-F (should be roughly 8-10 total) and rank them into one list, highest priority first.
2. Your ranking should weigh **both** severity score and real-world exploitability (Part F) — a Critical-rated finding you can't actually act on may rank below a Medium-rated one you've already proven is exploitable. State your ranking logic in one sentence.

Part J — Scenario: Re-Prioritizing for Business Context

The exact same findings list means something different depending on who you hand it to. Below are two client profiles. You are not rescanning anything — you're deciding whether your Part I ranking would change if this were the real target.

Client 1 — NorthPeak Credit Union: A financial services company. This server processes loan applications containing Social Security numbers and bank account details. It sits in a PCI-DSS-scoped segment and has an SLA requiring any Critical finding to be remediated within 24 hours.

Client 2 — Downstream Labs: An internal developer sandbox used to test code before it reaches a separate, isolated production environment. No real customer data ever touches this box, and it's rebuilt from a template weekly regardless of what's found on it.

1. Re-rank your Part I list once for each client.
2. For at least two findings, write one sentence explaining why its priority moved (or deliberately didn't) between the two contexts.
3. **Reflection (2-3 sentences):** CVSS scores are supposed to be objective. Does this exercise suggest they're sufficient on their own for prioritization, or is CVSS actually just one input into a decision that also needs business context?

6. Write the Vulnerability Assessment Report

Unlike your Lab 3/4 penetration-test write-ups, which centered on one exploited finding told as a narrative, a vulnerability assessment report is built for **breadth**. Produce:

1. A findings table covering every item from Part I: finding, severity, affected host, one-line evidence, one-line fix
2. Your Part E credentialed-vs-uncredentialed comparison, as its own short section
3. Your Part G false-positive investigation and verdict, as its own short section
4. Your Part H remediation-verification record, as its own short section
5. **Two** executive summaries — one for NorthPeak Credit Union, one for Downstream Labs — each two sentences, reflecting your Part J re-prioritization
6. One paragraph explicitly distinguishing this deliverable from a penetration-test report — who would receive each one, and why they're not interchangeable

7. What to Submit

1. Part A — your CVSS band notes
2. Part B — five Nikto findings with your initial severity guesses
3. Part C — your triage table and reflection answer
4. Part D — your OpenVAS top-five findings with CVSS scores (or a note that you skipped this optional part and why)
5. Part E — your credentialed-vs-uncredentialed comparison and reflection (or a note that you skipped it)
6. Part F — your vsftpd cross-validation and your manual-verification judgment call
7. Part G — your false-positive investigation, verdict, and reasoning
8. Part H — your remediation-verification record
9. Part I — your prioritized list and ranking rationale
10. Part J — both re-ranked lists and your reflection
11. Section 6 — the full vulnerability assessment report

8. Troubleshooting

Symptom	Likely cause / fix
<code>nikto</code> not found	<code>sudo apt install nikto</code>
OpenVAS/Greenbone web UI won't load	The service can take several minutes to start after <code>gvm-start</code> — wait and retry before troubleshooting further
OpenVAS feed sync seems stuck	It genuinely can take a long time on first run (sometimes over an hour) — this is normal; start it at the beginning of a session with time to spare
Credentialed scan reports the same results as uncredentialed	Confirm the SSH credential actually saved correctly on the target configuration — a failed credential falls back to network-only checks without always saying so clearly
<code>ssh msfadmin@192.168.56.102</code> is refused	Confirm Metasploitable has fully booted, and that you haven't already hardened SSH in an earlier remediation step without reverting the snapshot
Nikto returns very few findings	Confirm you're pointing it at the right IP and port — try adding <code>-p 80,8080</code> explicitly if the web service runs on a non-default port

This is Lab 7 in the course sequence. It feeds directly into Lab 8's SQL Injection lab, where you'll go deep on one specific class of finding this lab would otherwise only flag at the surface — and where the same false-positive and business-context judgment calls from Parts G and J come up again in a different form.